

SmartAdmin 《Vue3规范 V3.0》

阅读须知

在阅读《Java规范》之前，请一定先阅读上一篇《高质量代码思想 V3.0》!!!，[前往阅读](#)

一、Vue3 基础规范

1.1、项目命名

全部采用小写方式，以中划线分隔。

```
1  正例：`smart-admin`
2  反例：`mall_management-system / mallManagementSystem`
```

javascript

1.2、目录、文件命名

目录、文件名 均以 小写方式，以中划线分隔。

```
1  正例：`/head-search/`、`/shopping-car/`、`smart-logo.png`、`role-form.vue`
2  反例：`/headSearch/`、`smartLogo.png`、`RoleForm.vue`
```

javascript

1.3、单引号、双引号、分号

- html中、vue的template 中 标签属性 使用 **双引号**
- 所有js中的 字符串 使用 **单引号**
- 所有js中的代码行换行要用 **分号**

二、Vue3 组合式 API 规范

2.1、使用setup语法糖

- 组件必须使用 `setup` 语法糖
- `setup` 大法方便简洁
- 全局都要使用`setup`语法糖

2.2、组合式Composition API 规范

组件内必须使用模块化思想，把代码 进行 拆分；

参照 vue3官方文档对于 Composition Api的理解：[更灵活的代码组织](#)，组合式Api，即 Composition API 解决的是让 相互关联的代码在一起，以更方便的组织代码，故代码格式如下：

即：将相关的 变量和代码 写到一起，并使用 行注释 进行分块

js

```
1  <script setup>
2  // 各种需要导入
3  import xxxxx;
4  import xxxxx;
5  import xxxxx;
6
7  // ----- 定义组件属性和对外暴露的方法、以及抛出的事件 -----
8
9  // ----- 表格查询的 变量和方法 -----
10
11 // ----- 批量操作的 变量和方法 -----
12
13 // ----- 表单的 变量和方法 -----
14
15 </script>
```

举例，分成了两个模块，即：

- 模块1：显示、隐藏操作的 变量和方法
- 模块2：表单的 变量和方法

【以下是正确的例子】

```
1  <script setup>
2    import { ref, reactive } from 'vue';
3    import { message } from 'ant-design-vue';
4    import { SmartLoading } from '@/components/framework/smart-loading';
5    import _ from 'lodash';
6    import { categoryApi } from '@/api/business/category/category-api';
7    import { smartSentry } from '@/lib/smart-sentry';
8
9    // emit
10   const emit = defineEmits('reloadList');
11
12   // 组件
13   const formRef = ref();
14
15   // ----- 显示、隐藏操作的 变量和方法 -----
16
17   // 是否展示抽屉
18   const visible = ref(false);
19   // 显示
20   function showModal(categoryType, parentId, rowData) {
21     Object.assign(form, formDefault);
22     form.categoryType = categoryType;
23     form.parentId = parentId;
24     if (rowData && !_.isEmpty(rowData)) {
25       Object.assign(form, rowData);
26     }
27     visible.value = true;
28   }
29   // 隐藏
30   function onClose() {
31     Object.assign(form, formDefault);
32     visible.value = false;
33   }
34
35   // ----- 表单的 变量和方法 -----
36   // 查询表单默认值
37   const formDefault = {
38     categoryId: undefined, //分类id
39     categoryName: '', //分类名字
40     categoryType: 1, // 分类类型
41     parentId: undefined, // 父级id
42     disabledFlag: false, //是否禁用
43   };
```

```
44 // 查询表单
45 let form = reactive({ ...formDefault });
46 // 表单校验规则
47 const rules = {
48   categoryName: [{ required: true, message: '请输入分类名称' }],
49 };
50
51 function onSubmit() {
52   formRef.value
53     .validate()
54     .then(async () => {
55       SmartLoading.show();
56       try {
57         if (form.categoryId) {
58           await categoryApi.updateCategory(form);
59         } else {
60           await categoryApi.addCategory(form);
61         }
62         message.success(`${form.categoryId ? '修改' : '添加'}成功`);
63         emit('reloadList', form.parentId);
64         onClose();
65       } catch (error) {
66         smartSentry.captureError(error);
67       } finally {
68         SmartLoading.hide();
69       }
70     })
71     .catch((error) => {
72       console.log('error', error);
73       message.error('参数验证错误，请仔细填写表单数据!');
74     });
75 }
76
77 defineExpose({
78   showModal,
79 });
80 </script>
```

【错误的例子】

```
1 <script setup>
2   import { ref, reactive } from 'vue';
```

js

```
3   import { message } from 'ant-design-vue';
4   import { SmartLoading } from '@/components/framework/smart-loading';
5   import _ from 'lodash';
6   import { categoryApi } from '@/api/business/category/category-api';
7   import { smartSentry } from '@/lib/smart-sentry';
8
9   // ----- 定义 所有变量 -----
10
11  // emit
12  const emit = defineEmits('reloadList');
13  // 组件
14  const formRef = ref();
15  // 是否展示抽屉
16  const visible = ref(false);
17  // 查询表单默认值
18  const formDefault = {
19    categoryId: undefined, //分类id
20    categoryName: '', //分类名字
21    categoryType: 1, // 分类类型
22    parentId: undefined, // 父级id
23    disabledFlag: false, //是否禁用
24  };
25  // 查询表单
26  let form = reactive({ ...formDefault });
27  // 表单校验规则
28  const rules = {
29    categoryName: [{ required: true, message: '请输入分类名称' }],
30  };
31
32  // ----- 定义 所有方法 -----
33  // 显示
34  function showModal(categoryType, parentId, rowData) {
35    Object.assign(form, formDefault);
36    form.categoryType = categoryType;
37    form.parentId = parentId;
38    if (rowData && !_isEmpty(rowData)) {
39      Object.assign(form, rowData);
40    }
41    visible.value = true;
42  }
43  // 隐藏
44  function onClose() {
45    Object.assign(form, formDefault);
46    visible.value = false;
```

```
47     }
48
49     function onSubmit() {
50         formRef.value
51             .validate()
52             .then(async () => {
53                 SmartLoading.show();
54                 try {
55                     if (form.categoryId) {
56                         await categoryApi.updateCategory(form);
57                     } else {
58                         await categoryApi.addCategory(form);
59                     }
60                     message.success(`${form.categoryId ? '修改' : '添加'}成功`);
61                     emit('reloadList', form.parentId);
62                     onClose();
63                 } catch (error) {
64                     smartSentry.captureError(error);
65                 } finally {
66                     SmartLoading.hide();
67                 }
68             })
69             .catch((error) => {
70                 console.log('error', error);
71                 message.error('参数验证错误, 请仔细填写表单数据!');
72             });
73     }
74
75     defineExpose({
76         showModal,
77     });
78 </script>
```

vue 官方希望是 模块化的组织代码, 你看记得官方这个图吗?

Options API



Composition API



第一种写法，所有变量写到了一起，就像vue2的 data区域，所有方法写到一起，就像vue2中的 methods区域，这样下来其实和vue2没什么区别，也就语法稍微变了一下，那么就与 composition api的初衷违背了，下面引用官方的话

处理相同逻辑关注点的代码被强制拆分在了不同的选项中，位于文件的不同部分。在一个几百行的大组件中，要读懂代码中的一个逻辑关注点，需要在文件中反复上下滚动，这并不理想。另外，如果想要将一个逻辑关注点抽取重构到一个可复用的工具函数中，需要从文件的多个不同部分找到所需的正确片段。

SmartAdmin中的写法才真正做到了vue3组合式composition api的意图，每一块业务的变量与方法写到一起，比如上面代码中的 显示与隐藏 的变量和方法放到了一起， 表单 的变量和方法放到了一起，下面引用官方的话

现在与同一个逻辑关注点相关的代码被归为了一组：无需再为了一个逻辑关注点在不同的选项块间来回滚动切换。此外，现在可以很轻松地将这一组代码移动到一个外部文件中，不再需要为了抽象而重新组织代码，大大降低了重构成本，这在长期维护的大型项目中非常关键。

2.3 模板引用变量Ref

对于vue3中的模板引用ref，即 ref 是作为一个特殊的 attribute

```
1 <input ref="inputRef">
```

要求:

- 使用 ref方法, 参数为空 进行声明变量
- 变量必须以 Ref 为结尾
- template中的ref 也必须以 Ref 为结尾

比如上面的例子, 声明如下

```
1 const inputRef = ref();
```

js

2.4 变量和方法的注释

在使用Composition Api进行代码编写时, 有效的组织了代码, 但是由于Composition Api变量和方法会写到一起, 这时候注释就变得很有必要 要求:

- 变量必须都加上注释
- 方法必须加上注释 比如

```
1 // 查询 公告 默认值
2 const queryFormState = {
3   noticeTypeId: undefined, //分类
4   keywords: '', //标题、作者、来源
5   documentNumber: '', //文号
6   createUserId: undefined, //创建人
7   deletedFlag: undefined, //删除标识
8   createTimeBegin: null, //创建-开始时间
9   createTimeEnd: null, //创建-截止时间
10  publishTimeBegin: null, //发布-开始时间
11  publishTimeEnd: null, //发布-截止时间
12  pageNum: 1,
13  pageSize: PAGE_SIZE,
14 };
15 // 查询 公告 请求表单
16 const queryForm = reactive({ ...queryFormState });
```

js

三、Vue3 组件规范

3.1、组件文件名

组件文件名应该为 pascal-case 格式

正例：

1 components

2 | - my-component.vue

javascript

反例：

1 components

2 | - myComponent.vue

3 | - MyComponent.vue

javascript

3.2、父子组件文件名

和父组件紧密耦合的子组件应该以父组件名作为前缀命名 正例：

1 components

2 | - todo-list.vue

3 | - todo-list-item.vue

4 | - todo-list-item-button.vue

5 | - user-profile-options.vue （完整单词）

javascript

反例：

1 components

2 | - TodoList.vue

3 | - TodoItem.vue

4 | - TodoButton.vue

5 | - UProfOpts.vue （使用了缩写）

javascript

3.3、组件属性

组件属性较多，应该主动换行。

正例：

html

```
1 <MyComponent foo="a" bar="b" baz="c"
2   foo="a" bar="b" baz="c"
3   foo="a" bar="b" baz="c"
4 />
```

反例：

html

```
1 <MyComponent foo="a" bar="b" baz="c" foo="a" bar="b" baz="c" foo="a" bar="b" baz="c" />
```

3.4、模板中表达式

组件模板应该只包含简单的表达式，复杂的表达式则应该重构为计算属性或方法。复杂表达式会让你的模板变得不那么声明式。应该尽量描述应该出现的是什么，而非如何计算那个值。而且计算属性和方法使得代码可以重用。

正例：

js

```
1 <template>
2   <p>{{ normalizedFullName }}</p>
3 </template>
4
5 // 复杂表达式已经移入一个计算属性
6 computed: {
7   normalizedFullName: function () {
8     return this.fullName.split(' ').map(function (word) {
9       return word[0].toUpperCase() + word.slice(1)
10    }).join(' ')
11   }
12 }
```

反例：

js

```
1 <template>
2   <p>
```

```
3      {{
4          fullName.split(' ').map(function (word) {
5              return word[0].toUpperCase() + word.slice(1)
6          }).join(' ')
7      }}
8  </p>
9  </template>
```

3.5、标签顺序

单文件组件应该总是让标签顺序保持为 `<template>` 、`<script>`、`<style>`

正例：

```
1  <template>...</template>
2  <script>...</script>
3  <style>...</style>
```

java

反例：

```
1  <template>...</template>
2  <style>...</style>
3  <script>...</script>
```

四、Vue Router 规范

4.1、页面传参

页面跳转，例如 A 页面跳转到 B 页面，需要将 A 页面的数据传递到 B 页面，推荐使用 路由参数进行传参，即 `{query:param}`

正例：

```
1  let id = ' 123';
2  this.$router.push({ name: 'userCenter', query: { id: id } });
```

js

4.2、path 和 name 命名规范

- path `kebab-case` 命名规范（尽量与vue文件的目录结构保持一致，因为目录、文件名都是 `kebab-case`，这样很方便找到对应的文件）
- path 必须以 / 开头，即使是children里的path也要以 / 开头。如下示例
- 经常有这样的场景：某个页面有问题，要立刻找到这个vue文件，如果不用以/开头，path为parent和children组成的，可能经常需要在router文件里搜索多次才能找到，而如果以/开头，则能立刻搜索到对应的组件
- name 命名规范采用 `KebabCase` 命名规范且和component组件名保持一致！（因为要保持keep-alive特性，keep-alive按照component的name进行缓存，所以两者必须高度保持一致）

js

```
1 // 动态加载
2 export const reload = [
3   {
4     path: '/reload',
5     name: 'reload',
6     component: Main,
7     meta: {
8       title: '动态加载',
9       icon: 'icon iconfont'
10    },
11
12    children: [
13      {
14        path: '/reload/smart-reload-list',
15        name: 'SmartReloadList',
16        meta: {
17          title: 'SmartReload',
18          childrenPoints: [
19            {
20              title: '查询',
21              name: 'smart-reload-search'
22            },
23            {
24              title: '执行reload',
25              name: 'smart-reload-update'
26            },
27            {
28              title: '查看执行结果',
29              name: 'smart-reload-result'
30            }
31          ]
32        }
33      }
34    ]
35  }
36 ]
```

```
31         ]
32     },
33     component: () =>
34         import('@/views/reload/smart-reload/smart-reload-list.vue')
35     }
36 ]
37 }
38 ];
```

五、Vue 项目规范

5.1、目录规范

1	src	源码目录
2	-- api	所有api接口
3	-- assets	静态资源, images, icons, styles等
4	-- components	公用组件
5	-- config	配置信息
6	-- constants	常量信息, 项目所有Enum, 全局常量等
7	-- directives	自定义指令
8	-- i18n	国际化
9	-- lib	外部引用的插件存放及修改文件
10	-- mock	模拟接口, 临时存放
11	-- plugins	插件, 全局使用
12	-- router	路由, 统一管理
13	-- store	vuex, 统一管理
14	-- theme	自定义样式主题
15	-- utils	工具类
16	-- views	视图目录
17	-- role	role模块名
18	-- -- role-list.vue	role列表页面
19	-- -- role-add.vue	role新建页面
20	-- -- role-update.vue	role更新页面
21	-- -- index.less	role模块样式
22	-- -- components	role模块通用组件文件夹
23	-- employee	employee模块

5.2、api 目录

- api文件要以api为结尾，比如 `employee-api.js`、`login-api.js`，方便查找
- api文件必须导出对象必须以 `Api` 为结尾，如：`employeeApi`、`noticeApi`
- api中以一个对象将方法包裹
- api中的注释，必须和后端 swagger 文档保持一致，同时保留后端作者

正例：

前端：`department-api.js`

js

```
1  import { getRequest, postRequest } from '@lib/axios';
2
3  export const departmentApi = {
4    /**
5     * @description: 查询部门列表 @author 卓大
6     * @param {*}
7     * @return {*}
8     */
9    queryAllDepartment: () => {
10      return getRequest('/department/listAll');
11    },
12
13    /**
14     * @description: 查询部门树形列表 @author 卓大
15     * @param {*}
16     * @return {*}
17     */
18    queryDepartmentTree: () => {
19      return getRequest('/department/treeList');
20    },
21
22    /**
23     * @description: 添加部门 @author 卓大
24     * @param {*}
25     * @return {*}
26     */
27    addDepartment: (param) => {
28      return postRequest('/department/add', param);
29    },
30
31    /**
32     * @description: 更新部门信息 @author 卓大
33     * @param {*}
34     * @return {*}
35     */
36  }
```

```
35     */
36     updateDepartment: (param) => {
37         return postRequest('/department/update', param);
38     }
};
```

5.3、assets 目录

assets 为静态资源，里面存放 images, styles, icons 等静态资源，静态资源命名格式为 kebab-case

```
1 | assets
2 | -- icons
3 | -- images
4 |   |-- background-color.png
5 |   |-- upload-header.png
6 | -- styles
```

5.4、components 目录

此目录应按照组件进行目录划分，目录命名为 kebab-case，一个组件必须一个单独的目录；目的：

- 一个组件一个目录是为了将来组件的扩展，因为这是整个项目公用的组件
- 组件入口必须为 index.vue，原因也是因为这是整个项目公用的组件

举例如下：

```
1 | components
2 | -- error-log
3 |   |-- index.vue
4 |   |-- index.less
5 | -- markdown-editor
6 |   |-- index.vue
7 |   |-- index.js
8 | -- kebab-case
```

5.5、constants 目录

此目录存放项目所有常量和枚举，如果常量在 vue 中使用，请使用 [src/plugin/smart-enums-plugin.js](#) 插件，也可以使用第三方库：[vue-enum 插件](#)，但是还不支持vue3，等待更新吧

具体要求：

- 常量文件要以 `const` 为结尾，比如 `login-const.js` 、 `file-const.js`
- 变量要：大写下划线，比如 `LOGIN_RESULT_ENUM` 、 `LOGIN_SUCCESS` 、 `LOGIN_FAIL`
- 如果是 枚举，变量必须以 `ENUM` 为结尾，如：`LOGIN_RESULT_ENUM` 、
`CODE_FRONT_COMPONENT_ENUM`

目录结构：

```
1 | constants
2 | -- index-const.js
3 | -- role-const.js
4 | -- employee-const.js
```

例子： `employee-const.js`

js

```
1 export const EMPLOYEE_STATUS = {
2   NORMAL: {
3     value: 1,
4     desc: '正常'
5   },
6   DISABLED: {
7     value: 1,
8     desc: '禁用'
9   },
10  DELETED: {
11    value: 2,
12    desc: '已删除'
13  }
14 };
15
16 export const EMPLOYEE_ACCOUNT_TYPE = {
17   QQ: {
18     value: 1,
19     desc: 'QQ登录'
20   },
21   WECHAT: {
22     value: 2,
```

```
23     desc: '微信登录'
24   },
25   DINGDING: {
26     value: 3,
27     desc: '钉钉登录'
28   },
29   USERNAME: {
30     value: 4,
31     desc: '用户名密码登录'
32   }
33 };
34
35 export default {
36   EMPLOYEE_STATUS,
37   EMPLOYEE_ACCOUNT_TYPE
38 };
```

5.6、router 与 store 目录

这两个目录一定要将业务进行拆分，不能放到一个 js 文件里。

router 尽量按照 views 中的结构保持一致

store 按照业务进行拆分不同的 js 文件

5.7、views 目录

目录要求，按照模块划分，其中具体文件名要求如下：

- 如果是列表页面，要以list为结尾，如 `role-list.vue` 、 `cache-list.vue`
- 如果是 表单页面，要以 form为结尾，如 `role-form.vue` 、 `notice-add-form.vue`
- 如果是 modal弹窗，要以 modal为结尾，如 表单弹窗 `role-form-modal.vue` ， 详情 `role-detail-modal.vue`
- 如果是 drawer 抽屉页面，要同上以 `Drawer` 为结尾

```
1  |-- views
2  |   |-- role
3  |   |   |-- role-list.vue
4  |   |   |-- role-add-form.vue
5  |   |   |-- role-update-form-modal.vue
6  |
```

视图目录
role模块名
role列表页面
role新建页面
role更新页面

js

7				-- index.less	role模块样式
8				-- components	role模块通用组件文件夹
9				-- role-title-modal.vue	role弹出框组件
10				-- employee	employee模块
11				-- behavior-log	行为日志log模块
				-- code-generator	代码生成器模块

联系我们

[1024创新实验室-主任: 卓大](#)，混迹于各个技术圈，研究过计算机，熟悉点 java，略懂点前端。

[1024创新实验室](#)，卓大的软件公司，致力于成为中原领先、国内一流的技术团队，以AI+数字化为驱动，用技术为产业互联网提供无限可能，业务如下：

- 供应链（网络货运、大宗贸易进销存ERP、物流TMS、B2B电商、仓储WMS、AI提效等）
- 教育（就业创业大数据平台、继续教育平台、在线教育系统、题库等）
- AI+软件（软件定制外包、数据大屏、国产化改造、人员外包、技术顾问、技术培训等）
- 欢迎各类合作哦~

		 “写好代码少加班，快乐工作，热爱生活，感谢支持。” 卓大的赞赏码	 @六边形工程师 抖音号：1024lab 卓大，1024创新实验室主任；
加微信：卓大 拉你入群，一起学习	公众号：六边形工程师 分享：赚钱、代码、生活	请“1024创新实验室” 烩面里加肉 咖啡配胡辣汤,提神又饱腹	抖音：六边形工程师 直播：赚钱、代码、中医

上一篇
[高质量代码思想 V3.0](#)

下一篇
[Java规范 V3.0](#)

